LangGraph: From Zero to Production-Ready

Phase 1 — Core Concepts (Sessions 1-3)

Session 1: State + Nodes. Session 2: Edges + Conditional Routing. Session 3: First LLM Node.

Phase 2 — Real Patterns (Sessions 4-6)

Session 4: Loops + Termination. Session 5: Tool Use. Session 5.5: MCP. Session 6: Multi-Agent Coordination.

Phase 3 — Production Patterns (Sessions 7-9)

Session 7: Persistence + Memory. Session 8: Human-in-the-Loop. Session 9: Streaming.

Phase 4 — Production Stack (Sessions 10-13)

Session 10: FastAPI Integration. Session 11: Observability (LangSmith). Session 12: MCP with Real Tools. Session 13: Docker + Deployment.

Final Project — Customer Support Multi-Agent System

Phase 1 (Complete): Supervisor + specialist agents, MCP tools, FastAPI SSE streaming, Postgres persistence.

Phase 2 (Complete): PostgreSQL, streamable-http MCP in separate container, docker-compose, EC2 deployment.

Phase 3 — Remaining Work

1. Human-in-the-Loop on Billing

Pause billing agent before responding, wait for human approval. interrupt_after=[billing_agent] + /approve/{thread_id} endpoint. Real use case: refunds over need human sign-off.

2. Per-Agent MCP Servers

Split single MCP server into billing-tools-server and technical-tools-server. Different teams own different tool servers. Each agent connects to its own MCP server at startup.

3. Auto-Retry from Checkpoint

On agent failure: get_state_history() -> find last good checkpoint -> invoke(None, retry_config). Long-running workflows should not restart from zero on failure.

4. Evaluation Pipeline

25-case dataset per agent. Exact match evaluator for supervisor routing. LLM-as-judge for answer quality. Tool selection eval. Run with LangSmith evaluate(). Baseline -> change prompt -> rerun -> compare. Three levels: unit evals, integration evals, production evals on real traffic.

5. Agentic RAG

Ingest documents into FAISS or Pinecone vector store. RAG tool does semantic search. Expose via MCP so agents call it as a tool. Agent decides when to search and how to use results. Covers vector databases gap on every AI JD.

Production Stack Architecture

```
curl / Frontend
    -> FastAPI (POST /chat, GET /history, POST /approve)
        -> LangGraph Multi-Agent
            -> Supervisor -> Billing / Technical / General agents
                -> MCP Tool Servers (per agent)
                    -> Tavily web search, RAG vector store
        -> PostgreSQL (persistence)
        -> LangSmith (observability + evaluation)
Docker + EC2 (deployment)
```

JD Coverage After Phase 3

Graphs/routing/loops, multi-agent coordination, tool use + MCP, persistence + memory, streaming, FastAPI, observability, Docker + deployment, human-in-the-loop, per-agent MCP, auto-retry, evaluation pipelines, agentic RAG / vector databases.

Everything on the BofA Junior Gen AI Engineer JD — built, not just read about.